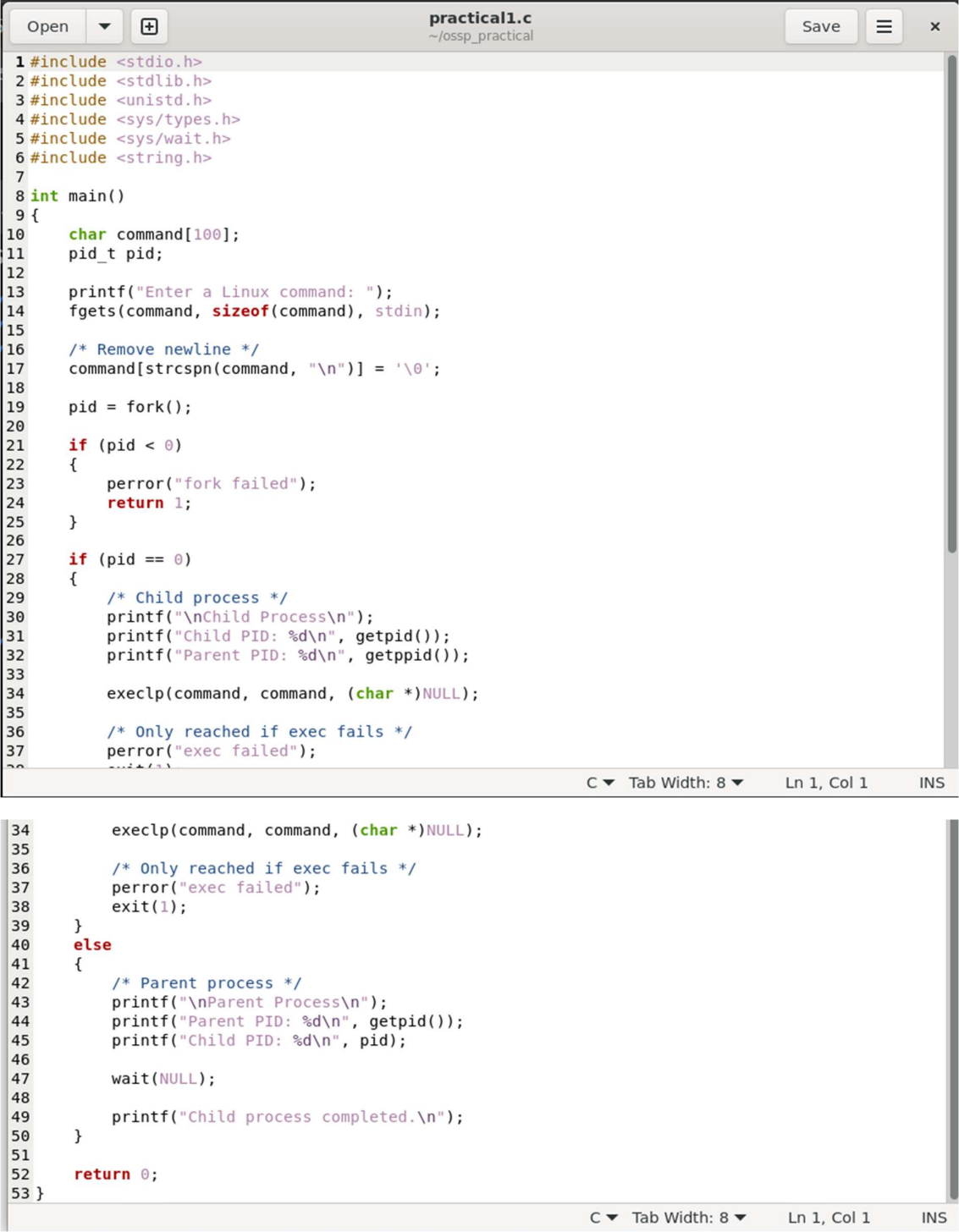


Practical – 1 – OS



The image shows a code editor window titled "practical1.c" with the file path "~/ossp_practical". The code is a C program that demonstrates process creation using `fork()` and `exec()`. It includes headers for `stdio.h`, `stdlib.h`, `unistd.h`, `sys/types.h`, `sys/wait.h`, and `string.h`. The `main` function prompts the user to enter a Linux command, removes the trailing newline, and then forks a child process. The child process executes the command using `execvp`. If `execvp` fails, it prints an error and exits. The parent process waits for the child to complete and then prints a message indicating the child process has completed.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6 #include <string.h>
7
8 int main()
9 {
10     char command[100];
11     pid_t pid;
12
13     printf("Enter a Linux command: ");
14     fgets(command, sizeof(command), stdin);
15
16     /* Remove newline */
17     command[strcspn(command, "\n")] = '\0';
18
19     pid = fork();
20
21     if (pid < 0)
22     {
23         perror("fork failed");
24         return 1;
25     }
26
27     if (pid == 0)
28     {
29         /* Child process */
30         printf("\nChild Process\n");
31         printf("Child PID: %d\n", getpid());
32         printf("Parent PID: %d\n", getppid());
33
34         execvp(command, command, (char *)NULL);
35
36         /* Only reached if exec fails */
37         perror("exec failed");
38         exit(1);
39     }
40     else
41     {
42         /* Parent process */
43         printf("\nParent Process\n");
44         printf("Parent PID: %d\n", getpid());
45         printf("Child PID: %d\n", pid);
46
47         wait(NULL);
48
49         printf("Child process completed.\n");
50     }
51
52     return 0;
53 }
```

Practical – 1 – OS

```
Saketh@SakethVutharavalli:~$ mkdir -p ~/ossp_practical
cd ~/ossp_practical
Saketh@SakethVutharavalli:~/ossp_practical$ gedit practical1.c
Saketh@SakethVutharavalli:~/ossp_practical$ gcc practical1.c -o practical1
Saketh@SakethVutharavalli:~/ossp_practical$ ./practical1
Enter a Linux command: ls

Parent Process
Parent PID: 552
Child PID: 553

Child Process
Child PID: 553
Parent PID: 552
practical1 practical1.c
Child process completed.
Saketh@SakethVutharavalli:~/ossp_practical$ gedit practical1.c
█
```